

CNN とは：全結合層から畳み込みへ

Dive into Deep Learning 輪読ゼミ

山口 海

June 3, 2026

参考: D2L-ja 7.1, https://d2l-jp.me/chapter_convolutional-neural-networks/why-conv.html

1 7.1. CNN とは: 全結合層から畳み込みへ

- 7.1.1. 不変性
- 7.1.2. MLP への制約
 - 7.1.2.1. 平行移動不変性
 - 7.1.2.2. 局所性
- 7.1.4. チャンネル
- 7.1.5. 要約と考察

2 7.2. 画像のための畳み込み

- 7.2.1. 相互相関演算
- 7.2.3. 画像における物体のエッジ検出
- 7.2.4 カーネルの学習

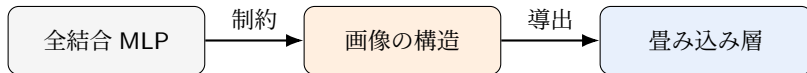
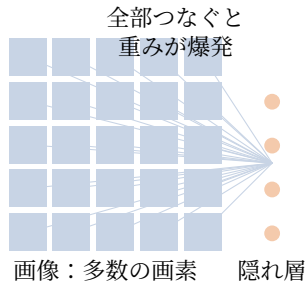
3 7.3. パディングとストライド

7.1. CNN とは: 全結合層から畳み込みへ

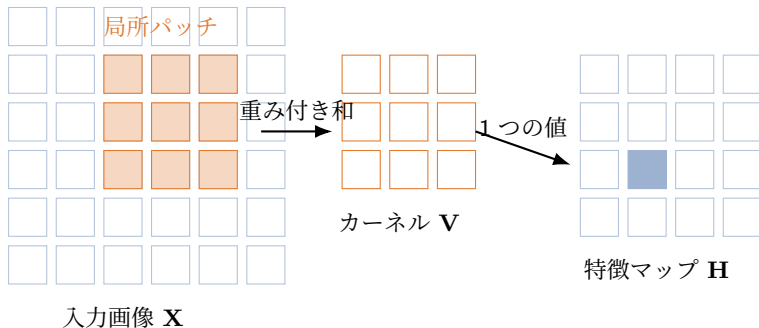
問い

なぜ画像認識では、全結合層ではなく畳み込み層を使うのか？

- ① 画像は高次元で、そのまま全結合にするとパラメータが爆発する。
- ② しかし画像には、位置関係や局所構造という強い規則性がある。
- ③ その規則性を帰納バイアスとしてモデルに入れる。
- ④ その結果として、自然に畳み込みが現れる。



畳み込み層の計算イメージ



1つの出力値の意味

入力画像の小さなパッチとカーネルの「一致度」を測ったスコアと考えられる。

CNN に入れる 2つの帰納バイアス

1. 平行移動等価性 / 不変性

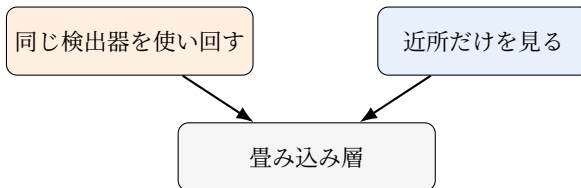
同じ局所パターンが画像のどこに現れても、同じように反応してほしい。

位置がずれる $\Rightarrow$ 特徴マップもずれる

2. 局所性

ある位置の特徴を計算するには、画像全体ではなく、その近くの画素だけ見ればよい。

近傍パッチ $\Rightarrow$ 局所特徴

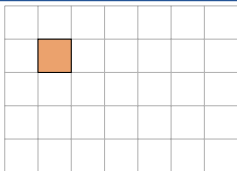


この節で理解したいこと

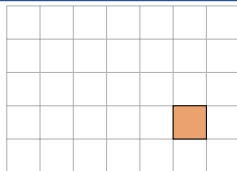
画像に MLP をそのまま使うとパラメータが多すぎる。しかし画像には、**平行移動に対する性質**と**局所性**がある。これらを MLP に制約として入れると、自然に**畳み込み層**が出てくる。

7.1.1. 不変性: どこにあっても同じものは同じ

猫, 縦線, 目のような特徴は, 画像の左上にあっても右下にあっても, 同じ種類の特徴として認識したい.



左上に特徴



右下に特徴

- 最終的な分類では, 多少位置が変わっても同じクラスと判断したい.
- 中間層では, 入力をずらすと特徴マップの反応もずれる, という性質が欲しい.

平行移動不変性

入力をずらしても, 出力が変わらない.

$$f(T\mathbf{X}) = f(\mathbf{X})$$

- 猫画像 $\mapsto$ 猫
- 右にずれた猫画像 $\mapsto$ 猫

平行移動等価性

入力をずらすと, 出力も同じようにずれる.

$$f(T\mathbf{X}) = Tf(\mathbf{X})$$

- 特徴が移動すると, 特徴マップ上の反応位置も移動する.

7.1.2. MLP への制約: 画像を 2 次元配列として扱う

ベクトル入力 of 全結合層

入力を $\mathbf{x} = (x_1, \dots, x_d)^\top$, 隠れ表現を $\mathbf{h} = (h_1, \dots, h_m)^\top$ とすると,

$$\mathbf{h} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

$$h_i = b_i + \sum_{k=1}^d W_{i,k} x_k.$$

- h_i は隠れ層の i 番目のユニット.
- x_k は入力の k 番目の成分.
- $W_{i,k}$ は「入力 k が隠れユニット i に与える重み」.

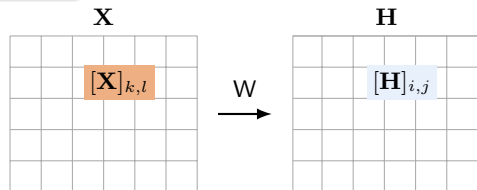
全結合層では, 各出力 $[\mathbf{H}]_{i,j}$ がすべての入力画素 $[\mathbf{X}]_{k,l}$ を見る.

ベクトルから行列へ

出力の位置が (i, j) , 入力の位置が (k, l) なので,

$$[\mathbf{H}]_{i,j} = [\mathbf{U}]_{i,j} + \sum_k \sum_l [\mathbf{W}]_{i,j,k,l} [\mathbf{X}]_{k,l}.$$

- ここでは簡単のため, $\mathbf{X}$ と $\mathbf{H}$ は同じ形状とする.
- $[\mathbf{U}]_{i,j}$ は位置 (i, j) のバイアス.



入力画像と隠れ表現を, 2 次元の行列として扱う.

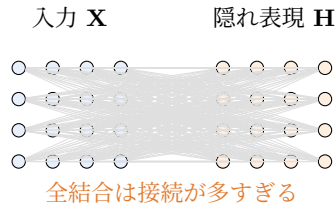
$$\mathbf{X} = ([\mathbf{X}]_{i,j}), \quad \mathbf{H} = ([\mathbf{H}]_{i,j})$$

添字を相対位置で書き直す

パラメータ数の爆発

全結合では,

$$\text{入力画素数} \times \text{出力ユニット数} = 10^6 \times 10^6 = 10^{12}.$$



入力位置を出力位置からのズレで表す

出力位置 (i, j) に対して, 入力位置 (k, l) を

$$k = i + a, \quad l = j + b$$

と書く. ここで (a, b) は出力位置から見た相対的なズレである.

$$\begin{aligned} [\mathbf{H}]_{i,j} &= [\mathbf{U}]_{i,j} + \sum_k \sum_l [\mathbf{W}]_{i,j,k,l} [\mathbf{X}]_{k,l} \\ &= [\mathbf{U}]_{i,j} + \sum_a \sum_b [\mathbf{V}]_{i,j,a,b} [\mathbf{X}]_{i+a,j+b}. \end{aligned}$$

対応関係

$$[\mathbf{V}]_{i,j,a,b} = [\mathbf{W}]_{i,j,i+a,j+b}.$$

7.1.2.1. 平行移動不変性

重み共有: 平行移動に対する制約

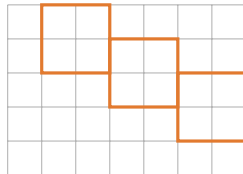
重みとバイアスが出力位置 (i, j) に依存しないと仮定する.

$$[\mathbf{V}]_{i,j,a,b} = [\mathbf{V}]_{a,b}, \quad [\mathbf{U}]_{i,j} = u.$$

すると,

$$[\mathbf{H}]_{i,j} = u + \sum_a \sum_b [\mathbf{V}]_{a,b} [\mathbf{X}]_{i+a,j+b}.$$

- $[\mathbf{V}]_{a,b}$ は相対位置 (a, b) だけで重みが決まる.
- 同じ特徴検出器を画像全体に適用する.
- これを**重み共有**と呼ぶ.



同じフィルタを使い回す

まだ残る問題

それでも、各位置で画像全体に近い広い範囲を見ている。そこで次に局所性を入れる。

7.1.2.2. 局所性を式に入れる

局所範囲だけを残す

ある Δ を決めて,

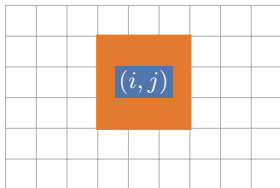
$$|a| > \Delta \quad \text{または} \quad |b| > \Delta \quad \implies \quad [\mathbf{V}]_{a,b} = 0$$

とする.

すると,

$$[\mathbf{H}]_{i,j} = u + \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} [\mathbf{V}]_{a,b} [\mathbf{X}]_{i+a,j+b}.$$

近傍だけを見る



- 相対位置 (a, b) が大きすぎるとき, 重みを 0 にする.
- エッジや模様は局所パッチから分かる.
- 遠く離れた画素との相互作用は, 深い層で段階的に扱えばよい.

7.1.4. チャネル: 画像は2次元ではなく3次元

これまでの単純化

これまでは、入力画像を2次元配列として

$$\mathbf{X} = ([\mathbf{X}]_{i,j})$$

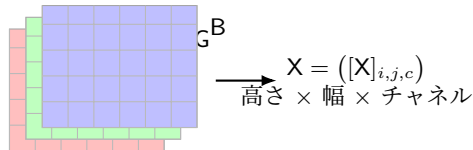
と書いていた。しかし、実際のカラー画像は赤、緑、青の3つのチャネルを持つ。

- 画像の縦方向: 高さ.
- 画像の横方向: 幅.
- 各画素における成分: チャネル.

$$\mathbf{X} \in \mathbb{R}^{H \times W \times C}$$

$$[\mathbf{X}]_{i,j,c}$$

は、位置 (i, j) のチャネル c の値.



複数入力チャネルの場合

入力が $[\mathbf{X}]_{i,j,c}$ のようにチャネル c を持つため、フィルタも $[\mathbf{V}]_{a,b,c}$ のようにチャネル方向の添字を持つ。したがって、空間方向 (a, b) だけでなくチャネル方向 c についても和を取る。

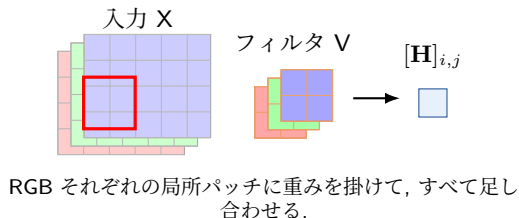
複数入力チャネルの畳み込み

1つの出力チャネルを作る場合

入力チャネル数を C_{in} とすると,

$$[H]_{i,j} = \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} \sum_{c=1}^{C_{in}} [V]_{a,b,c} [X]_{i+a,j+b,c}.$$

- a, b : 局所パッチ内の相対位置.
- c : 入力チャネル.
- $[V]_{a,b,c}$: 位置とチャネルごとの重み.



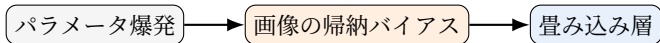
隠れ表現にもチャネルを持たせる

入力チャネルを c , 出力チャネルを d とすると,

$$[H]_{i,j,d} = \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} \sum_c [V]_{a,b,c,d} [X]_{i+a,j+b,c}.$$

7.1.5. 要約: CNN は第一原理から理解できる

- ① 画像を全結合層で扱うと、パラメータ数が大きくなりすぎる.
- ② 画像には、平行移動等価性と局所性という自然な構造がある.
- ③ これらを MLP に制約として課すと、畳み込み層が現れる.
- ④ 畳み込み層は、小さなカーネルを全位置で共有して特徴マップを作る.
- ⑤ 複数チャネルにより、多種類の局所特徴を同時に学習できる.

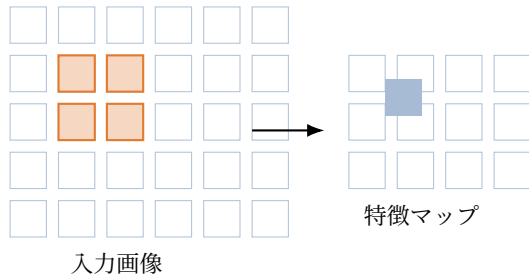


7.2. 画像のための畳み込み

この節で学ぶこと

前節では、画像に対して全結合層をそのまま使うのではなく、**局所性**と**重み共有**を使うと畳み込み層が自然に現れることを見た。この節では、実際に畳み込み層がどのように計算されるかを、コードと数値例で確認する。

- ① 相互相関演算を手で計算する。
- ② それを `corr2d` として実装する。
- ③ エッジ検出の例を見る。
- ④ カーネルをデータから学習する。
- ⑤ 特徴マップと受容野を理解する。

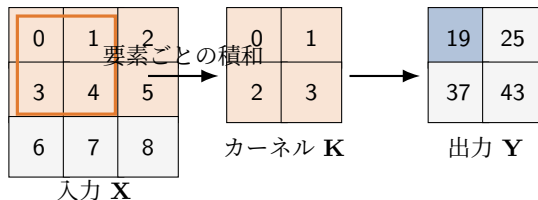


ここでの「畳み込み」は、深層学習の慣習では実際には**相互相関演算**を指すことが多い。厳密な数学用語とは少しずれるので注意する。

7.2.1. 相互相関演算：何を計算しているか

基本アイデア

小さなカーネルを入力画像の上でスライドさせる。各位置で、**重なった要素どうしを掛け算して足し合わせる**。その1つの値が出力の1画素になる。



$$0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3 = 19.$$

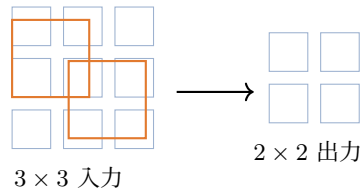
出力サイズ：なぜ少し小さくなるのか

入力サイズとカーネルサイズ

入力が $n_h \times n_w$ ，カーネルが $k_h \times k_w$ のとき，パディングなし・ストライド 1 の出力サイズは

$$(n_h - k_h + 1) \times (n_w - k_w + 1).$$

- カーネル全体が画像の中に収まる位置でしか計算できない。
- そのため，画像の端では計算位置が減る。
- 例： 3×3 入力に 2×2 カーネルを使うと，出力は 2×2 になる。



出力サイズを小さくしたくない場合は，次節で学ぶ**パディング**を使う。

corr2d の実装：出力サイズの準備

```
def corr2d(X, K): #@save
    """Compute 2D cross-correlation."""
    h, w = K.shape # kernel height and width

    # output size: (input size) - (kernel size) + 1
    Y = d2l.zeros((X.shape[0] - h + 1, X.shape[1] - w + 1))
```

何をしているか

入力 X とカーネル K を受け取り、カーネルのサイズ (h, w) を取り出す。
入力が 3×3 、カーネルが 2×2 なら、出力は

$$(3 - 2 + 1) \times (3 - 2 + 1) = 2 \times 2$$

になる。

corr2d の実装：各位置で内積を計算

```
# slide the kernel over all output positions
for i in range(Y.shape[0]):
    for j in range(Y.shape[1]):

        # elementwise product, then sum
        Y[i, j] = d2l.reduce_sum((X[i: i + h, j: j + w] * K))

return Y
```

何をしているか

$X[i: i+h, j: j+w]$ は、入力 X からカーネルと同じ大きさの部分を切り出している。
その部分と K を成分ごとに掛けて、すべて足すことで

$$Y_{i,j} = \sum_{a=0}^{h-1} \sum_{b=0}^{w-1} X_{i+a,j+b} K_{a,b}$$

を計算している。

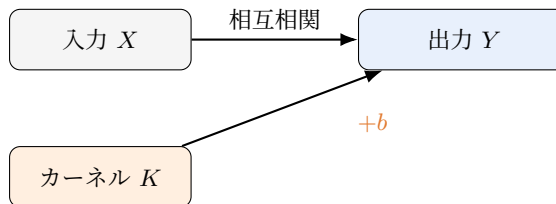
7.2.2 畳み込み層

畳み込み層の出力

畳み込み層は、入力とカーネルの相互相関を計算し、さらにバイアスを加える。

$$\text{出力} = \text{corr2d}(\text{入力}, \text{カーネル}) + \text{バイアス}.$$

- **カーネル**：局所パッチを見るための重み.
- **バイアス**：出力全体に足されるスカラー値.
- 学習では、カーネルとバイアスをデータから調整する.



$h \times w$ 畳み込みとは、カーネルの高さが h 、幅が w である畳み込み層のこと。

畳み込み層の実装

```
class Conv2D(nn.Module):
    def __init__(self, kernel_size):
        super().__init__()

        # learnable kernel
        self.weight = nn.Parameter(
            torch.rand(kernel_size))

        # learnable bias
        self.bias = nn.Parameter(torch.zeros(1))

    def forward(self, x):
        return corr2d(x, self.weight) + self.bias
```

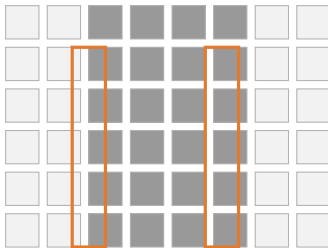
ポイント

- `weight` が畳み込みカーネル. ここではランダムに初期化している.
- `bias` は最初は 0.
- `forward` で `corr2d` を呼び出し, 最後にバイアスを足す.

7.2.3. エッジ検出：変化している場所を見つける

エッジとは

画像の中で画素値が急に変わる場所を**エッジ**と呼ぶ。ここでは、白を 1、黒を 0 とし、中央が黒い簡単な画像を考える。



白 → 黒, 黒 → 白 の境界がエッジ

7.2.3 画像における物体のエッジ検出

```
X = d2l.ones((6, 8))
X[:, 2:6] = 0

K = d2l.tensor([[1.0, -1.0]])

Y = corr2d(X, K)
Y
```

$$K = \begin{pmatrix} 1 & -1 \end{pmatrix}$$

は横に隣り合う画素の差を取るカーネルである.

説明

- X は 6×8 の白黒画像である.
- $X[:, 2:6] = 0$ により, 中央部分を黒くしている.
- 左右で画素値が同じなら, 差は 0 になる.
- $1 \rightarrow 0$ の境界では

$$1 \cdot 1 + 0 \cdot (-1) = 1$$

となる.

- $0 \rightarrow 1$ の境界では

$$0 \cdot 1 + 1 \cdot (-1) = -1$$

となる.

- したがって, このカーネルは縦方向の境界を検出する.

7.2.4 カーネルの学習

```
conv2d = nn.LazyConv2d(1, kernel_size=(1, 2),
                        bias=False)

X = X.reshape((1, 1, 6, 8))
Y = Y.reshape((1, 1, 6, 7))
lr = 3e-2

for i in range(10):
    Y_hat = conv2d(X)
    l = (Y_hat - Y) ** 2
    conv2d.zero_grad()
    l.sum().backward()
    conv2d.weight.data[:] -= lr * conv2d.weight.grad
```

説明

- 前節では $K = (1, -1)$ を手で決めた.
- ここでは, 入力 X と正解出力 Y からカーネルを学習する.
- Y_hat は畳み込み層の予測出力.
- $l = (Y_hat - Y) ** 2$ で二乗誤差を計算する.
- 誤差が小さくなるようにカーネルを更新する.

手で設計したエッジ検出カーネルも, データから学習できる.

```
d2l.reshape(conv2d.weight.data, (1, 2))
```

出力例

```
tensor([[ 0.9414, -1.0291]])
```

説明

- 学習後の重みは

$(0.94, -1.03)$

のようになる。

- これは、手で設定した

$K = (1, -1)$

に非常に近い。

- つまり、ニューラルネットワークは「エッジを検出するフィルタ」を自動で見つけたことになる。

CNN では、カーネルを人間が設計するのではなく、学習によって獲得する。

7.3 パディングとストライド

パディング

- 畳み込みを行うと、出力サイズは小さくなる.
- 境界の周りに 0 を追加すると、出力サイズを保ちやすい.
- これを**ゼロパディング**という.

$$(n_h - k_h + p_h + 1) \times (n_w - k_w + p_w + 1)$$

```
conv2d = nn.LazyConv2d(1, kernel_size=3,  
                        padding=1)  
comp_conv2d(conv2d, X).shape  
# torch.Size([8, 8])
```

ストライド

- カーネルを何マスずつ動かすかを**ストライド**という.
- 通常は 1 マスずつ動かす.
- ストライドを大きくすると、出力サイズを小さくできる.

$$\left\lfloor \frac{n_h - k_h + p_h + s_h}{s_h} \right\rfloor \times \left\lfloor \frac{n_w - k_w + p_w + s_w}{s_w} \right\rfloor$$

```
conv2d = nn.LazyConv2d(1, kernel_size=3,  
                        padding=1, stride=2)  
comp_conv2d(conv2d, X).shape  
# torch.Size([4, 4])
```

7.2-7.3 まとめ

7.2 畳み込み層

- 畳み込み層では、入力画像 X にカーネル K を重ねて相互相関演算を行う。
- 実際には

$$Y = X \star K + b$$

のように、相互相関の結果にバイアスを加える。

- カーネル K は、エッジなどの局所的な特徴を取り出すフィルタとして働く。
- CNN では、このカーネルを手で決めるのではなく、データから学習する。

7.3 パディングとストライド

- パディングは、入力の周囲に 0 を追加して、出力サイズの縮小を抑える操作である。
- ストライドは、カーネルを何マスずつ動かすかを定める量である。
- ストライドを大きくすると、出力サイズは小さくなる。

畳み込み層は「局所的な特徴を取り出す層」であり、パディングとストライドによって出力サイズを調整できる。